Gerard Chaparro Redondo

Activitat 4.A: La pantalla

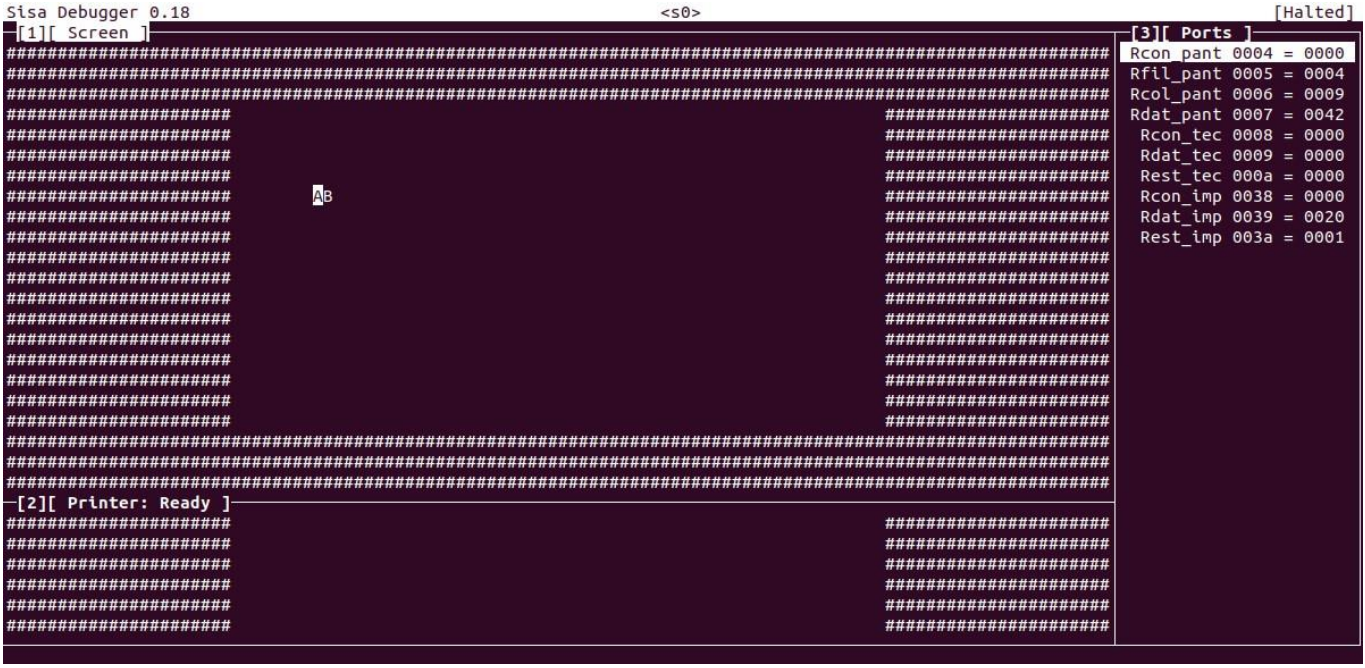
En aquesta activitat accedirem a la pantalla. Completa l'exercici 4.1 abans de continuar.
Exercici 4.1: Escriu un programa en alt nivell que mostri una 'A' a la posició [4,8] de la pantalla, amb atribut invers, i una 'B' a la posició [4,9] amb atribut normal. Observa que la fila és la mateixa en els dos casos, així que sols cal escriure-la un cop.

Resposta: En aquest exercici primer de tot he posat la dada de Rcon_pant al registre R3 per utilitzar-la més endavant, a continuació utilitzem el registre R1 per posar la dada de la fila i la columna (Rfil_pant i Rcol_pant). Després posem la lletra que volem mostrar per pantalla al registre R2 per posar-la a Rdat_pant y seguidament posem la dada de R3 a Rcon_pant per mostrar la lletra. I es repeteix el mateix procés per posar la segona lletra amb la columna posterior.

Codi:

```
.include "macros.s"
.include "crt0.s"

.data


.text
main:
        $movei R3,0x8000        ;
        movi R1,4               ;
        out Rfil_pant,R1        ;
        movi R1,8               ;
        out Rcol_pant,R1        ;
        $movei R2,0x0141        ;
        out Rdat_pant,R2        ;
        out Rcon_pant,R3        ;
        movi R1,9               ;
        out Rcol_pant,R1        ;
        $movei R2,0x0042        ;
        out Rdat_pant,R2        ;
        out Rcon_pant,R3        ;
        HALT                    ;
```

Prova de funcionament:



Activitat 4.B: El teclat

L'objectiu d'aquest apartat és provar l'accés al teclat sincronitzant-lo per enquesta. Completa l'exercici 4.2 abans de continuar**:**

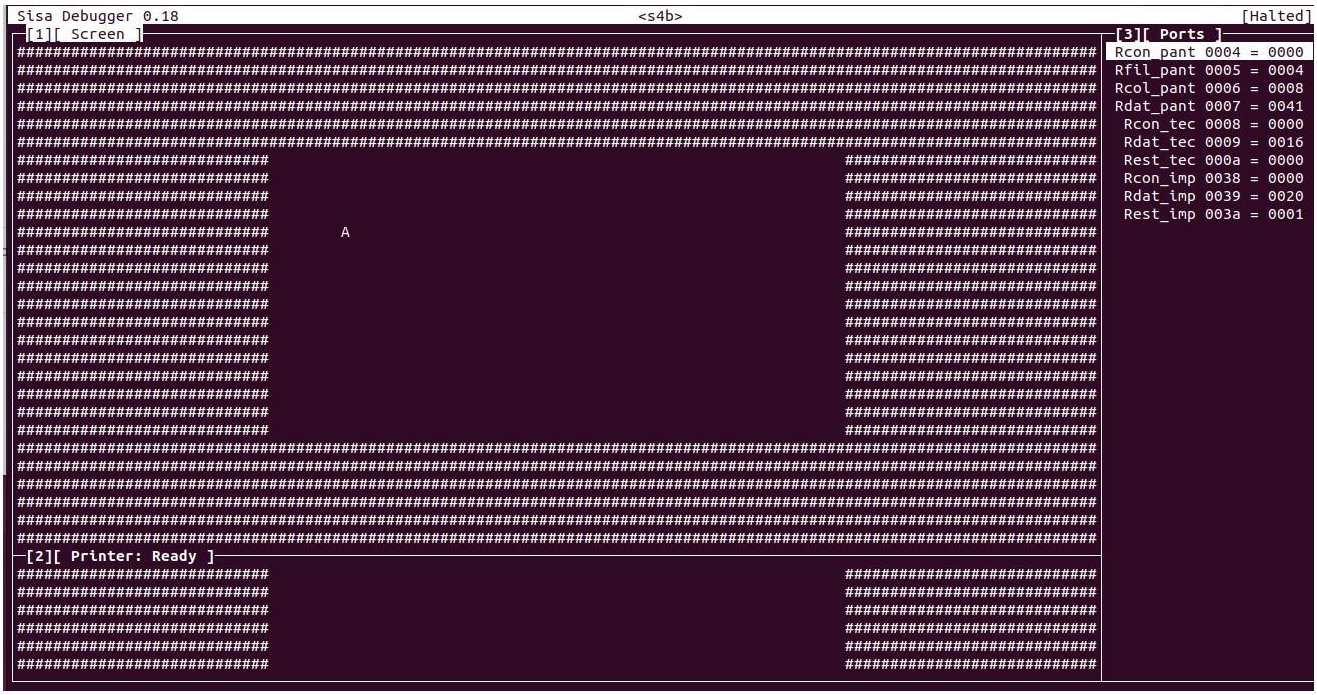
Exercici 4.2: Escriu un programa en alt nivell que esperi fins que es polsi una tecla qualsevol, i llavors escrigui una 'A' a la posició [4,8] de la pantalla, amb atribut normal.

Resposta: En aquest exercici primer de tot he de consultar tota la estona la variable Rest_tec per saber si l'usuari ha teclejat una tecla i fins que no es polsi una tecla no surt del bucle. A més hem de guardar la tecla teclejada per part de l'usuari per poder mostrar la lletra que ens demana el exercici, ja que així el sistema reconeix que la tecla deixa de estar pressionada. A continuació es repeteix el mateix procés de l'exercici 4.1 però ara només hem de mostrar una lletra 'A'.

Codi:

```
.include "macros.s"
.include "crt0.s"


.text

main:
do: IN R2,Rest_tec
    BZ R2,do
IN R1,Rdat_tec
movi R0,4
movi R1,8
out Rfil_pant,R0
out Rcol_pant,R1
movi R1,'A'
out Rdat_pant,R1
$movei R2,0xF000
out Rcon_pant,R2
HALT
```

Prova de funcionament:



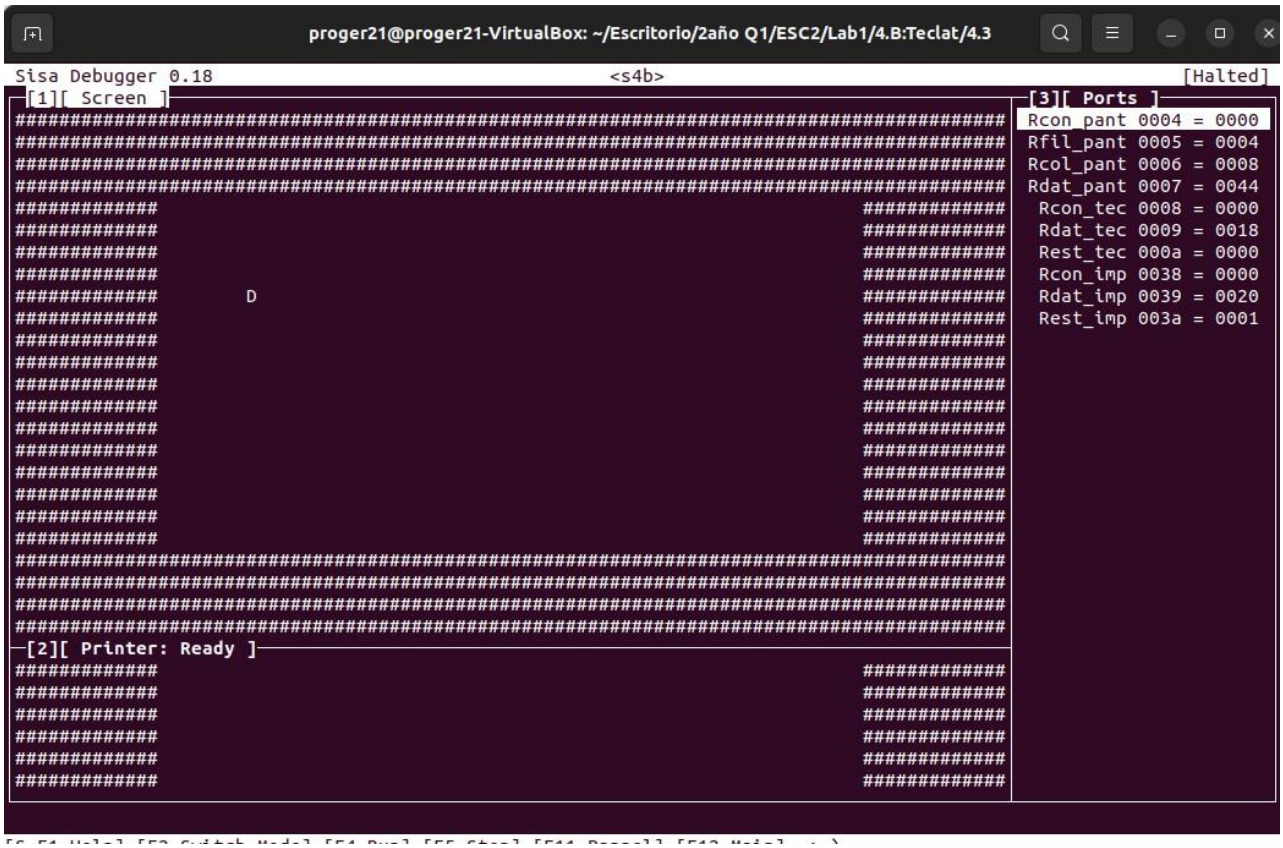
Exercici 4.3: Escriu una nova versió del programa en alt nivell anterior per tal que, en comptes d'una 'A', escrigui el caràcter associat a la tecla polsada. Fixa't que es tracta de fer sols un petit canvi en la finalització. Recorda que per traduir el codi de rastreig del teclat es disposa del vector tteclat. Ara, tradueix l'anterior codi a assemblador SISA-F en el mateix fitxer s4b.s(sols cal afegir les modificacions). Assembla'l i comprova que funciona, amb el simulador.

Resposta: En aquest exercici hem de fer el mateix procediment que a l'exercici anterior però la única diferencia es que la lletra que hem de mostrar per pantalla és la lletra que hem polsat. Per això hem de guardar la Rdat_tec(la dada de la tecla polsada) a un registre que en el meu cas es R1 i amb aquesta informació podem traduir aquesta dada amb el vector tteclat. I finalment la guardem a Rdat_pant.

Codi:

```
.include "macros.s"
.include "crt0.s"


.text

main:
do: IN R2,Rest_tec
    BZ R2,do
movi R0,4
movi R1,8
out Rfil_pant,R0
out Rcol_pant,R1
IN R1,Rdat_tec
$movei R3,tteclat
add R1,R3,R1
ldb R1,0(R1)
out Rdat_pant,R1
$movei R2,0xF000
out Rcon_pant,R2
HALT
```

Prova de funcionament:



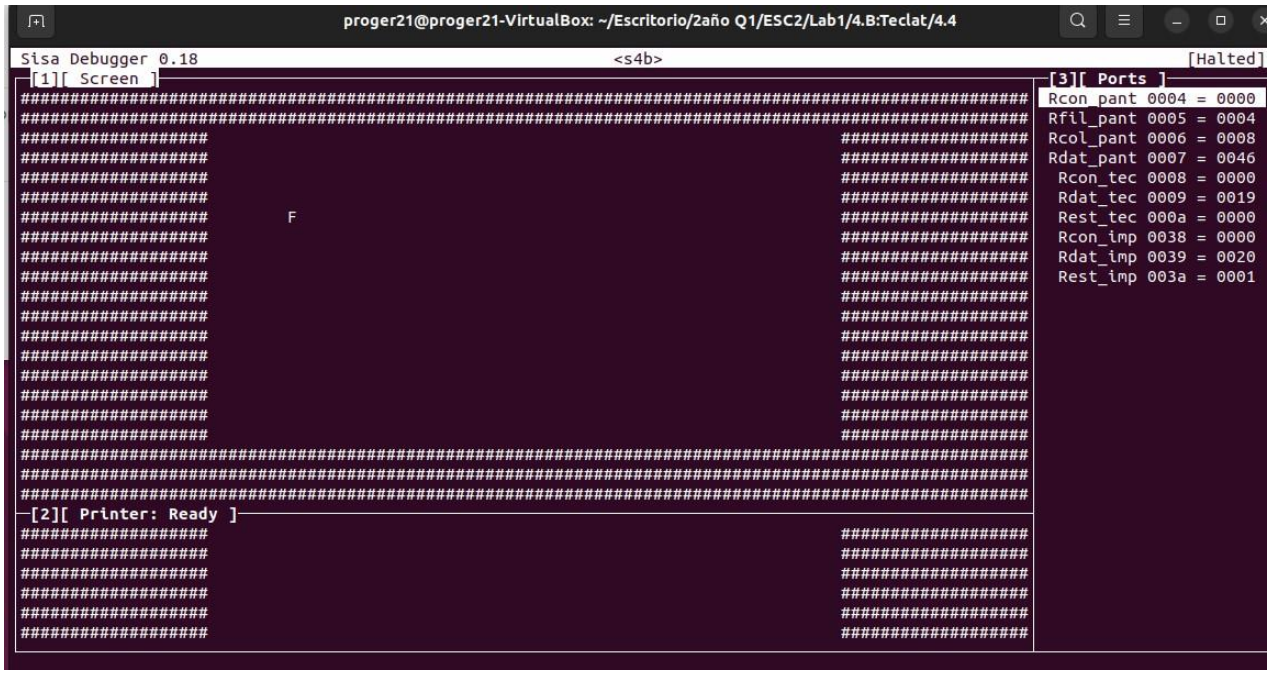
Exercici 4.4: Escriu en alt nivell una nova versió del programa anterior, tal que faci la mateixa tasca (llegir una tecla i escriure-la en pantalla) però repetidament, fins que la tecla polsada sigui una 'F'. Fixa't que es tracta sols d'insertar l'anterior programa dins un bucle.

Resposta: En aquest exercici hem de fer el mateix procediment que a l'exercici anterior però la única diferencia es que ho hem de fer repetidament amb un bucle fins que polsi 'F' l'usuari. Per això ens guardem la 'F' a R4 per poder comparar amb la R1 que es Rdat_tec en aquest cas i així saber si són la mateixa lletra. Aquesta comparació es fa al final del bucle després de mostrar la lletra teclejada per pantalla i si la comparació dona que no son iguals torna al principi del bucle i per una altra banda si son iguals es finalitza el progama.

Codi:

```
.include "macros.s"
.include "crt0.s"


.text

main: movi R4,'F'
do: IN R2,Rest_tec
    BZ R2,do
movi R0,4
movi R1,8
out Rfil_pant,R0
out Rcol_pant,R1
IN R1,Rdat_tec
$movei R3,tteclat
add R1,R3,R1
ldb R1,0(R1)
out Rdat_pant,R1
$movei R2,0xF000
out Rcon_pant,R2
XOR R0,R1,R4
BNZ R0,do
HALT
```

Prova de funcionament:



Activitat 4.C La impressora

L'objectiu d'aquest apartat és provar l'accés a la impressora sincronitzant-se per enquesta. Completa l'exercici 4.5 abans de continuar:

Exercici 4.5: Fes un programa en alt nivell que imprimeixi la paraula "Fi". Abans d'imprimir cada caràcter cal esperar que estigui preparada. I al final del programa, també.

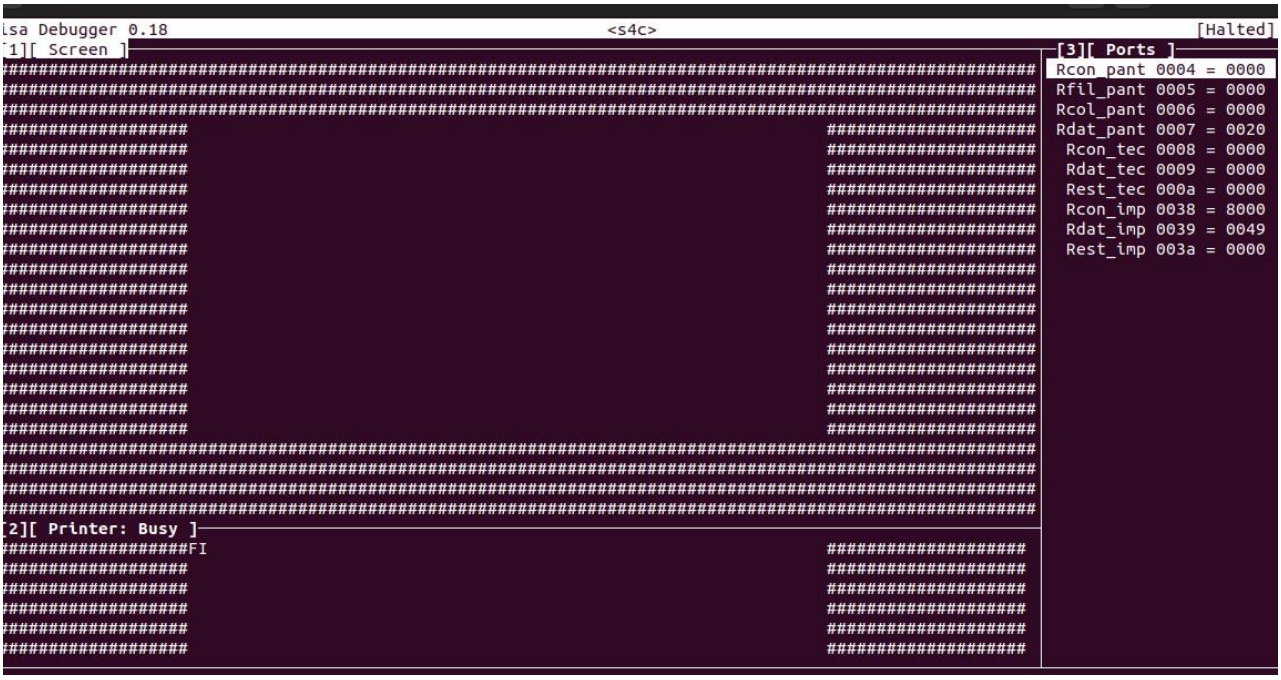
Resposta: En aquest exercici primer de tot fem un bucle que per sortir d'aquest l'estat de la impressora ha de estar "ready" (preparada). Després de sortir del bucle hem de imprimir la paraula "FI" i per fer això hem de imprimir lletra a lletra. El procés de imprimir una lletra es

guardar la lletra a R1 en el meu cas i ficar-la a Rdat_imp i a més posar la dada 0x8000 a Rcon_imp per a que es vegi en la impressora . Abans de cada impressió ha de haver un bucle per saber si la impressora esta preparada per això hi ha dos bucles al programa.

Codi:

```
.include "macros.s"
.include "crt0.s"


.data
.text

main:
while1: IN R1,Rest_imp
        BZ R1,while1
$Movei R1,'F'
OUT Rdat_imp,R1
$Movei R2,0x8000
OUT Rcon_imp,R2

while2: IN R1,Rest_imp
        BZ R1,while2
$Movei R1,'I'
OUT Rdat_imp,R1
$Movei R2,0x8000
OUT Rcon_imp,R2
HALT
```

Prova de funcionament:



Exercici 4.6: Escriu en alt nivell una nova versió del programa anterior que escrigui el vector frase a la impressora. Fixa't que el vector és un string, per tant, acaba amb un byte a zero (valor binari 0). Es tracta doncs de fer un bucle que iteri fins a trobar el zero final.

Resposta: En aquest exercici hem de fer un bucle per poder imprimir tota la frase i a més fer un bucle per saber si esta preparada la impressora . Per aquesta raó hem de guardar la direcció de memòria de char frase a un registre i llegir d' aquest registre el primer byte i al final del

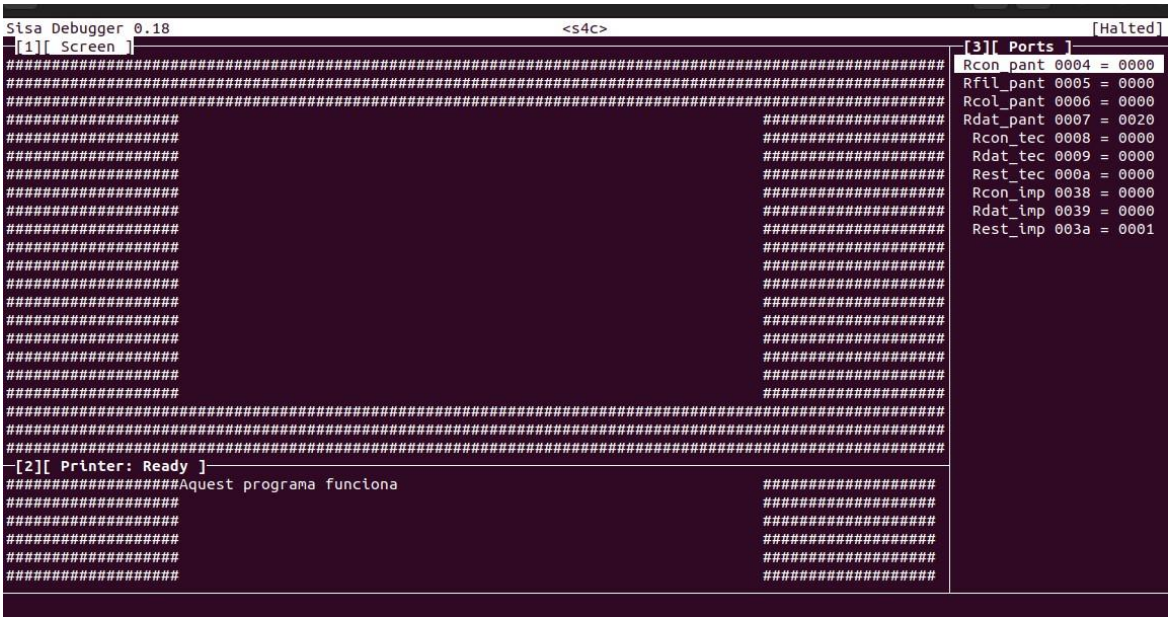
bucle hem de sumar un 1 al registre de la direcció de memòria per poder llegir la següent lletra.

Codi:

char frase[30] = "Aquest programa funciona";

```
.include "macros.s"
.include "crt0.s"

.data
frase: .asciz "Aquest programa funciona"
        .fill 5,1,0

.text
main:
$Movei R4,frase
Movi R5,0
bucle: Movi R2,30
CMPLE R6,R5,R2
BZ R6,Fi
LDB R3,0(R4)
while1: IN R1,Rest_imp
        BZ R1,while1
OUT Rdat_imp,R3
$Movei R2,0x8000
OUT Rcon_imp,R2
ADDI R4,R4,1
ADDI R5,R5,1
BNZ R6,bucle
Fi: HALT
```

Prova de funcionament:



Activitat 4.D

Visualització d'operacions de tractament de bits

L'objectiu d'aquest apartat és combinar l'accés a la pantalla, al teclat, i les operacions de tractament de bits, i ho farem en 3 parts:

primer lloc, escriurem un programa que mostri a les posicions [0,0]-[0,15] de la pantalla els 16 bits de la representació en binari (uns i zeros) de la variable w de mida word (figura 4.1),

mostrant el bit més significatiu a la columna 0 i el menys significatiu a la columna 15. El programa main fa una crida a la subrutina mostra passant-li com a paràmetre per valor la variable w. Aquesta subrutina no retorna res ja que és una acció, però mostra la representació binària del paràmetre i a les posicions [0,0]-[0,15]. Completa l'exercici 4.7 abans de seguir:

```
int w=0x8888;

main()

{

mostra(w);

}

void mostra(int i)

{

 register int col=16;

 out (Rfil_pant,0);

 do{

col--;

out(Rcol_pant,col);

out(Rdat_pant,'0'+(i&0x1));

out(Rcon_pant, 0x8000);

 i=i>>1;

 }while (col>0)

 }
```

Figura 4.1. Codi en alt nivell del programa main i de la subrutina mostra Exercici 4.7: Tradueix a SISA-F el programa main i la subrutina mostra de la figura 4.1

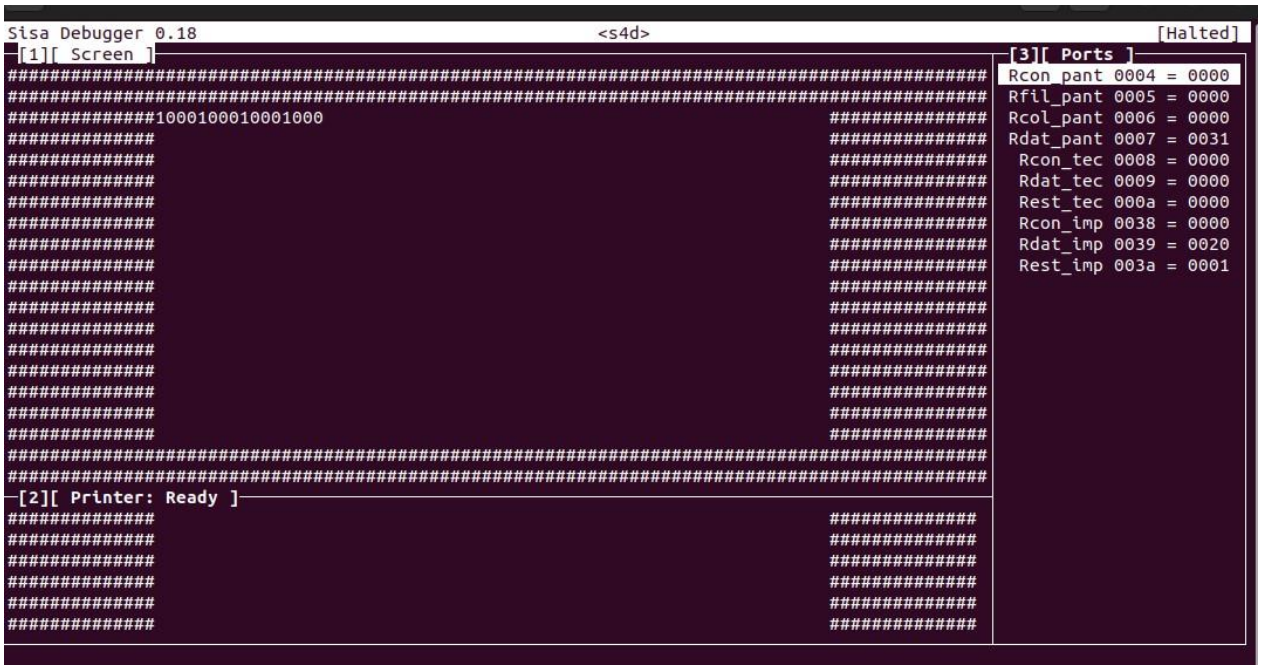
Exercici 4.7: Tradueix a SISA-F el programa main i la subrutina mostra de la figura 4.1.

Resposta: En aquest exercici fem un bucle on en cada iteració posem valors a Rcol_pant,Rdat_pant i Rcon_pant. En cada iteració la columna se li suma 1 més l'anterior, la dada que surt per pantalla la calculem fent un AND entre 0x1 i 'i' i sumant-li un '0'. Al final de cada iteració fem un desplaçament lògic de i(SHL) ,és a dir 1 lloc a l'esquerra.

Codi:

```
.include "macros.s"
.include "crt0.s"

.data
w: .word 0x8888
.text
main:$movei R1,w
LD R5,0(R1)
$call R6,mostra

mostra: $movei R1,16 ;R1->16 register int col = 16
movi R2,0
out Rfil_pant,R2
while:
ADDI R1,R1,-1        ;SUBI R1,R1,1
out Rcol_pant,R1
$movei R4,0x01
and R4,R5,R4
$movei R0,'0'
ADD R4,R4,R0
out Rdat_pant,R4
$movei R0,0x8000
out Rcon_pant,R0
movi R3,-1
sha R5,R5,R3    ;shl R5,R5,R3
BNZ R1,while
HALT
```

Prova de funcionament:



Escriu el codi anterior al fitxer s4d.s, i verifica que funcioni bé amb el simulador.

En segon lloc, volem extendre el programa anterior, perquè es comporti així:

1) Mostrar el valor de w (el codi del programa anterior).

2) Esperar fins que es polsi una tecla.

3) Modificar w segons quina sigui la tecla polsada:

• Si la tecla és una 'A', fer un desplaçament lògic de w (SHL), 1 lloc a l'esquerra.

• Si és una 'B', fer un desplaçament lògic de w (SHL), 1 lloc a la dreta.

• Si és una 'C', fer un desplaçament aritmètic de w (SHA), 1 lloc a la dreta.

• Si és una 'D', s'ha de dividir per dos l'enter w (DIV).

• Si és un número n (tecles '0'.... '9'), s'han de complementar els n bits de menys pes de w. Nota: aquesta operació es pot escriure en C així: w = wˆ((1<<< significa desplaçament a Esquerra.

4) Repetir des del pas número 1 (el programa no acaba, fa un bucle sense fi). Completa l'exercici 4.8 abans de continuar:

Exercici 4.8: Escriu en alt nivell (en C) el nou programa principal (main), suposant que l'acció mostra és la que està declarada a la figura 4.1.

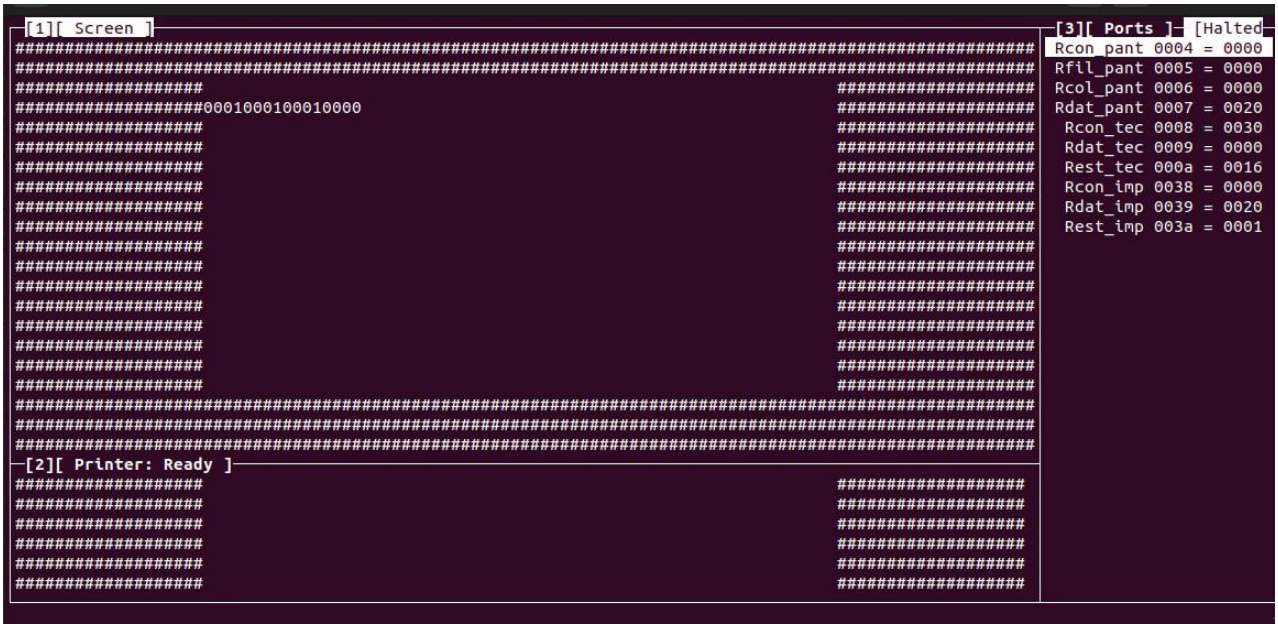
Resposta: En aquest exercici fem el mateix procés que l'exercici anterior excepte que hem de plantejar varies situacions on hem de canviar el valor 'i' depenent de la tecla polsada per part de l'usuari.

Codi:

```
.include "macros.s"
.include "crt0.s"

.data
w: .word 0x8888
.text
main:$movei R1,w
LD R5,0(R1)
do: IN R2,Rest_tec
    BZ R2,do
in R3,Rdat_tec
$call R6,mostra

mostra: $movei R1,16 ;R
movi R2,0
out Rfil_pant,R2
movi R0,0x16
$CMPEQ R0,R3,R0
BNZ R0,es_a
movi R0,0x24
$CMPEQ R0,R3,R0
BNZ R0,es_b
movi R0,0x22
$CMPEQ R0,R3,R0
BNZ R0,es_c
movi R0,0x18
$CMPEQ R0,R3,R0
BNZ R0,es_d
movi R0,9
$CMPLE R0,R3,R0
BNZ R0,n2
BZ R0,while
n2:
movi R0,0
$CMPLE R0,R0,R3
BZ R0,while
BNZ R0,es_n
while:
ADDI R1,R1,-1        ;S
out Rcol_pant,R1
$movei R4,1
and R4,R5,R4
$movei R0,'0'
ADD R4,R4,R0
out Rdat_pant,R4
$movei R0,0x8000
out Rcon_pant,R0
movi R0,-1
sha R5,R5,R0
BNZ R1,while
HALT

es_a:
```

```
es_a:
movi R0,1
shl R5,R5,R0
BNZ R0,while
es_b:
movi R0,-1
sha R5,R5,R0
BNZ R0,while
es_c:
movi R0,-1
sha R5,R5,R0
BNZ R0,while
es_d:
movi R0,2
DIV R5,R5,R0
BNZ R0,while
es_n:
movi R0,1
shl R0,R3,R0
ADDI R0,R0,-1
XOR R5,R5,R0
BNZ R0,while
```

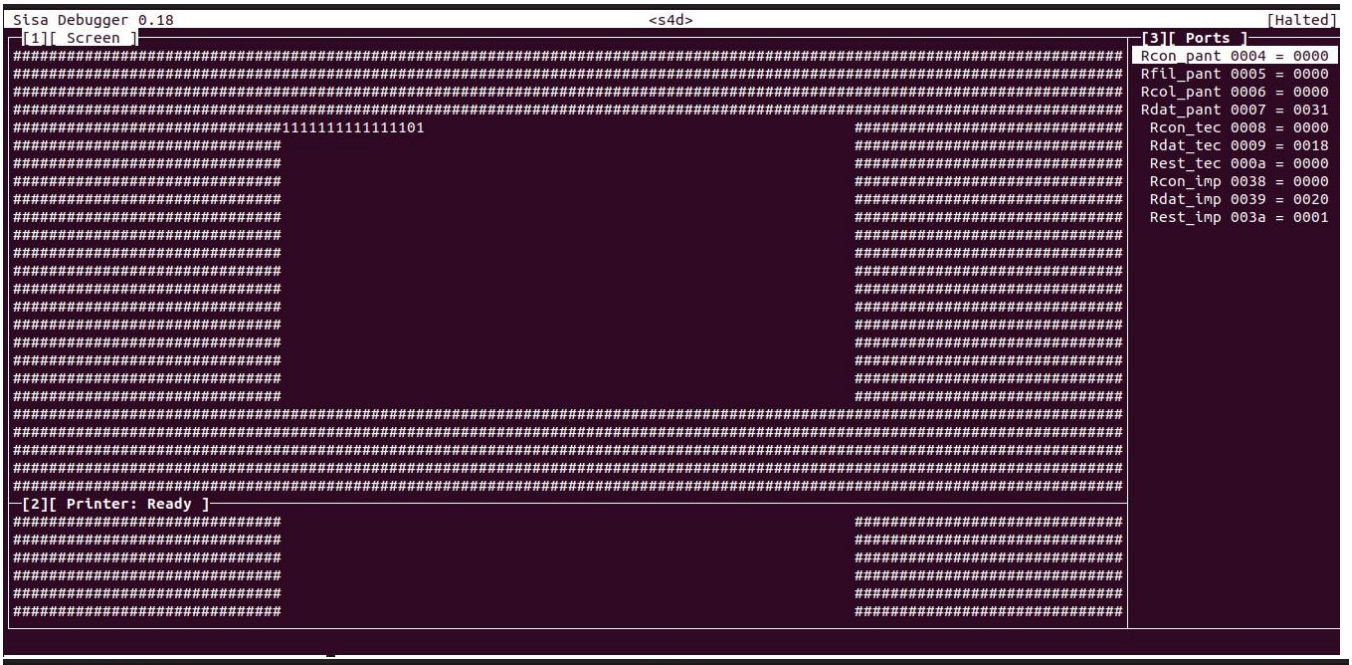
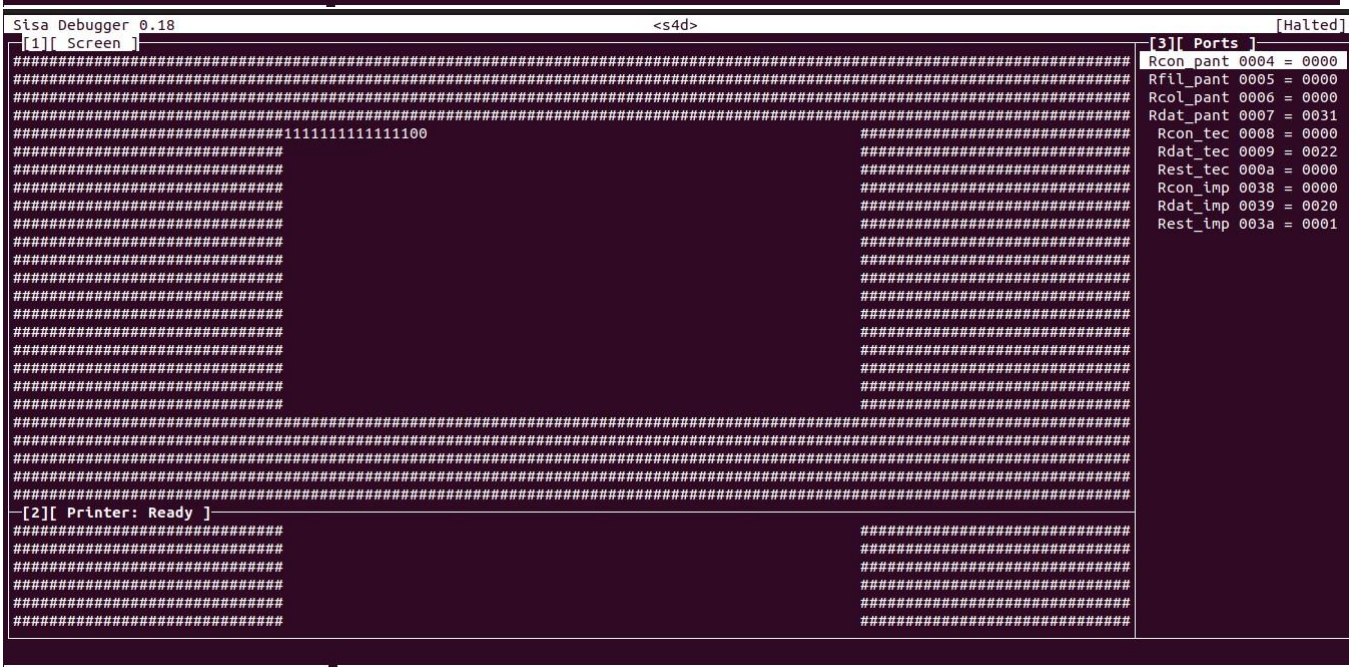
Prova de funcionament:



En tercer lloc volem provar el funcionament del programa amb un enter negatiu. Modifica la inicialització de w en el fitxer s4d.s per tal que w = 0xFFF9. Torna a assemblar i muntar el programa, i executa'l.

Exercici 4.9: Comprova que quan w és negatiu (w = 0xFFF9), la divisió DIV per 2 (tecla 'D') dona un resultat diferent que el desplaçament SHA una posició a la dreta (tecla 'C'). Per què?

Els valors de la tecla 'D' i de la tecla 'C' donen diferents , ja que alhora de fer un desplaçament SHA donarà un numero negatiu però probablement més cerca al 0 i mantenim el bit de signe

en el seu lloc. I si fem una divisió entre 2 donarà un numero negatiu però probablement el redunda cap a 0 en lloc de mantenir el signe en el seu lloc.

Proves de funcionament: